El Motor Lógico: Fundamentos y Decisiones en JavaScript

Dominando operadores, comparaciones y el control de flujo.

NotebookLM

La Anatomía de una Operación

Conceptos Clave:

- Operando: Los datos sin procesar.

- Unario: Un operador con un solo operando.

- Binario: Un operador con dos operandos.

-5 ← Operador Unario: invierte el signo

5 - 2 ← Operador Binario: resta un número de otro

Operandos

Insight: Formalmente, comparten el mismo símbolo, pero son operadores distintos según su contexto.

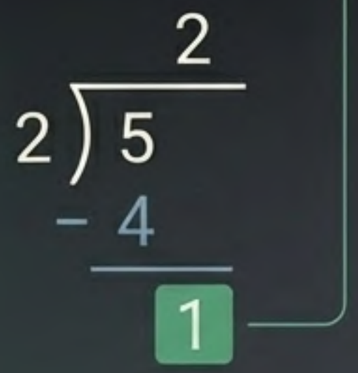
NotebookLM

Más Allá de la Escuela

El Resto (%)

No está relacionado con porcentajes. Devuelve el resto de la división entera.

```
5 % 2 === 1
```



La Exponenciación (**)

Eleva el primer operando a la potencia del segundo.

```
2 ** 3 === 8
```

Cubo

```
4 ** (1/2) === 2
```

Raíz Cuadrada

NotebookLM

El Operador Camaleón: Concatenación

La Regla de Oro: Si el operador '+' toca una cadena de texto (string), fusiona los valores. Si un operando es texto, el otro se convierte en texto.

Calculadora: Operación matemática pura

Pegamento: El número se convierte en texto

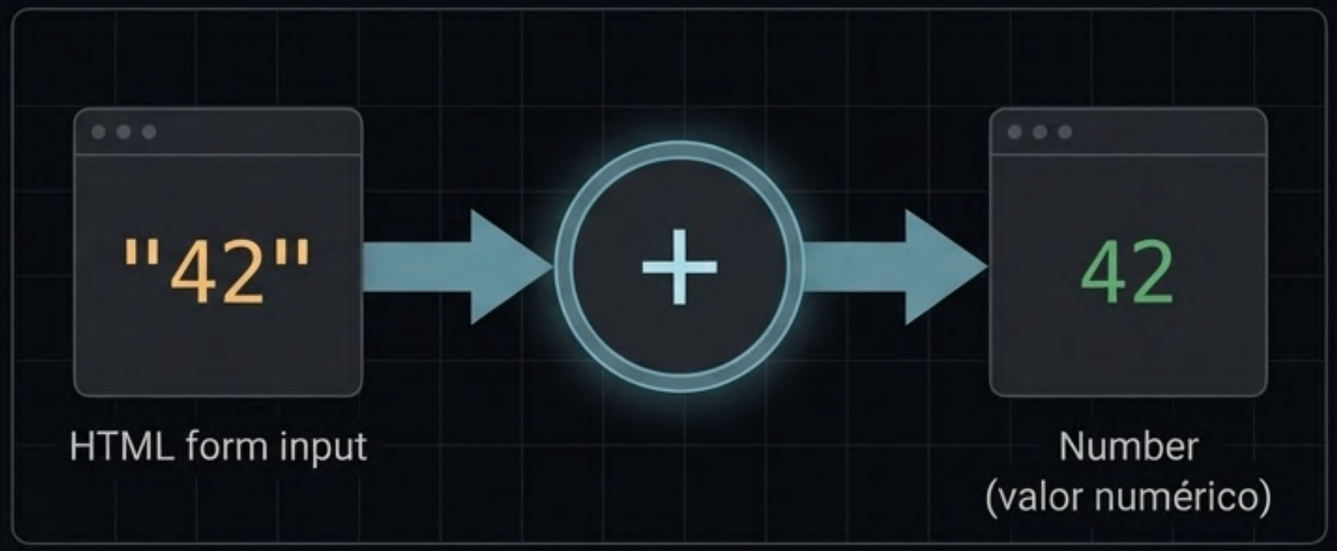
```
2 + 2 + '1' === '41' (Suma primero, concatena después)
'1' + 2 + 2 === '122' (Todo es tratado como texto)
```

NotebookLM

El Convertidor Instantáneo

Mecánica: El operador unario "+" no altera los números. Pero si operando es número, lo texto. Pero si el operando es texto, lo convierte en número instantáneamente.

Es la alternativa elegante a "Number()".



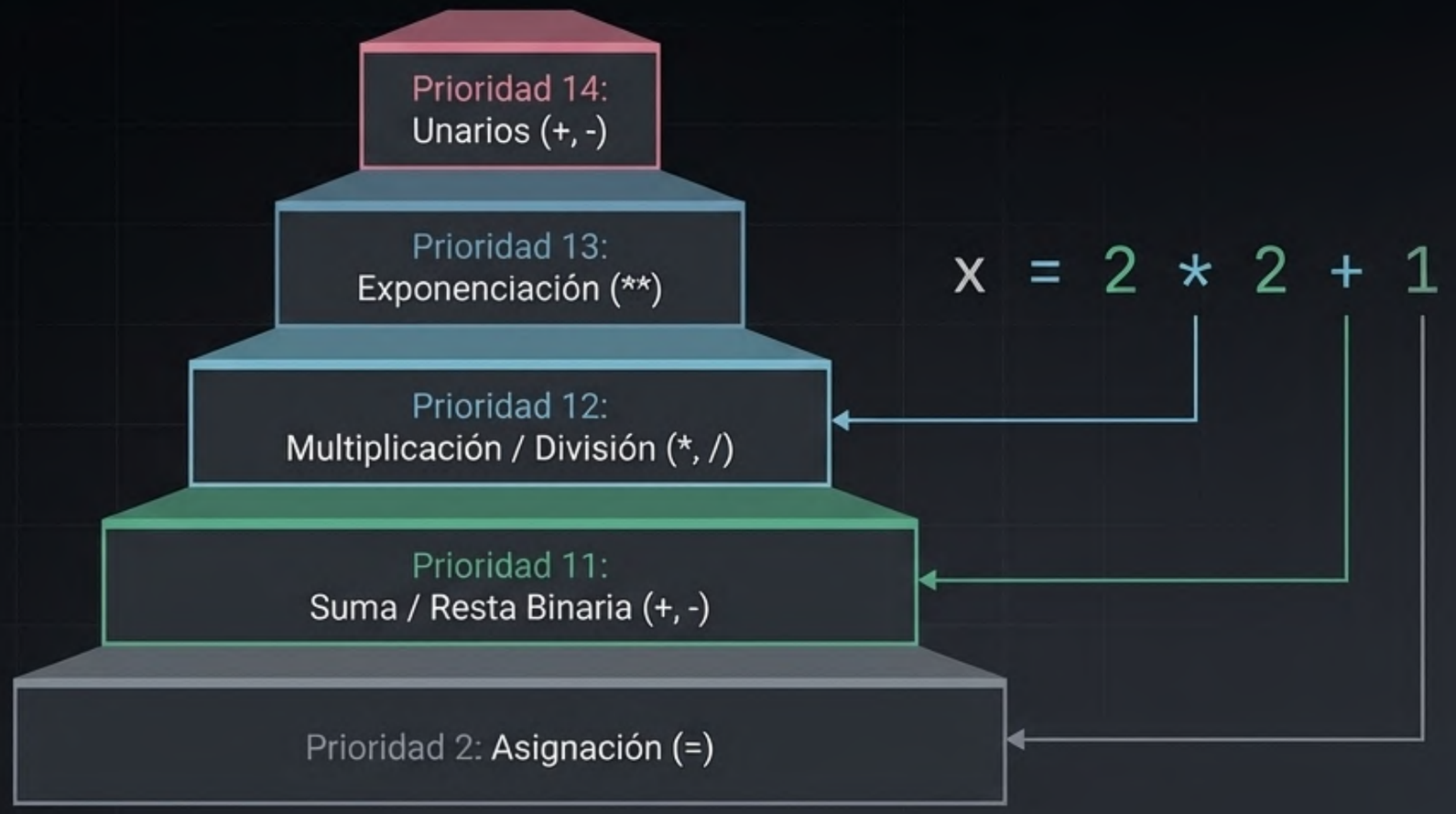
Uso Práctico

❌ (Bad)	`apples + oranges; // Devuelve '23'`
✅ (Good)	`+apples + +oranges; // Devuelve 5`

NotebookLM

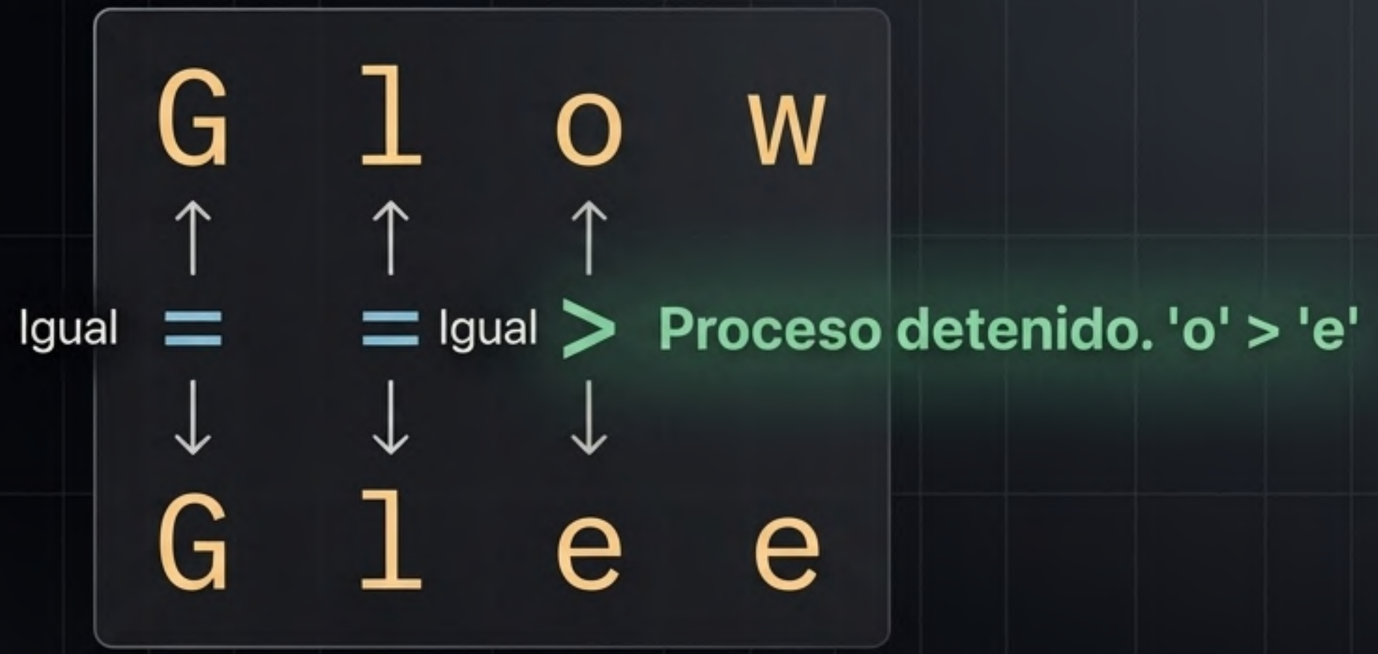
La Pirámide de Precedencia

El orden oculto: El motor sigue una jerarquía estricta. El número mayor se ejecuta primero.



NotebookLM

El Orden del Diccionario



Mecánica: JavaScript compara letra por letra usando la tabla Unicode.
Nota: Las mayúsculas tienen un índice menor que las minúsculas, por lo que 'a' > 'A' es true.

NotebookLM

La Trampa de la Igualdad

El problema con `'=='` es que convierte operandos a números antes de comparar. Un `'""'` y un `'false'` se convierten en 0.

Mecanismo	== (Igualdad Abstracta)	=== (Igualdad Estricta)
Conversión de Tipo	Sí (Peligroso) ⚠️	No (Comprueba valor y tipo) ✅
`0 == false`	`true`	`false`
`"" == false`	`true`	`false`

La Regla de la Industria: Usa siempre `'==='`.

NotebookLM

Fantasmas en la Máquina: null y undefined

La Pareja Dulce

En la igualdad abstracta (==), **null** y **undefined** solo son iguales entre sí.

```
null == undefined // true
```

Matemáticas Extrañas

En comparaciones matemáticas (<, >, >=), **null** se convierte en 0.

```
null > 0  // false
null == 0 // false

null == 0 // false (No
se convierte aquí)
null >= 0 // true
(Matemáticamente absurdo,
pero cierto)
```

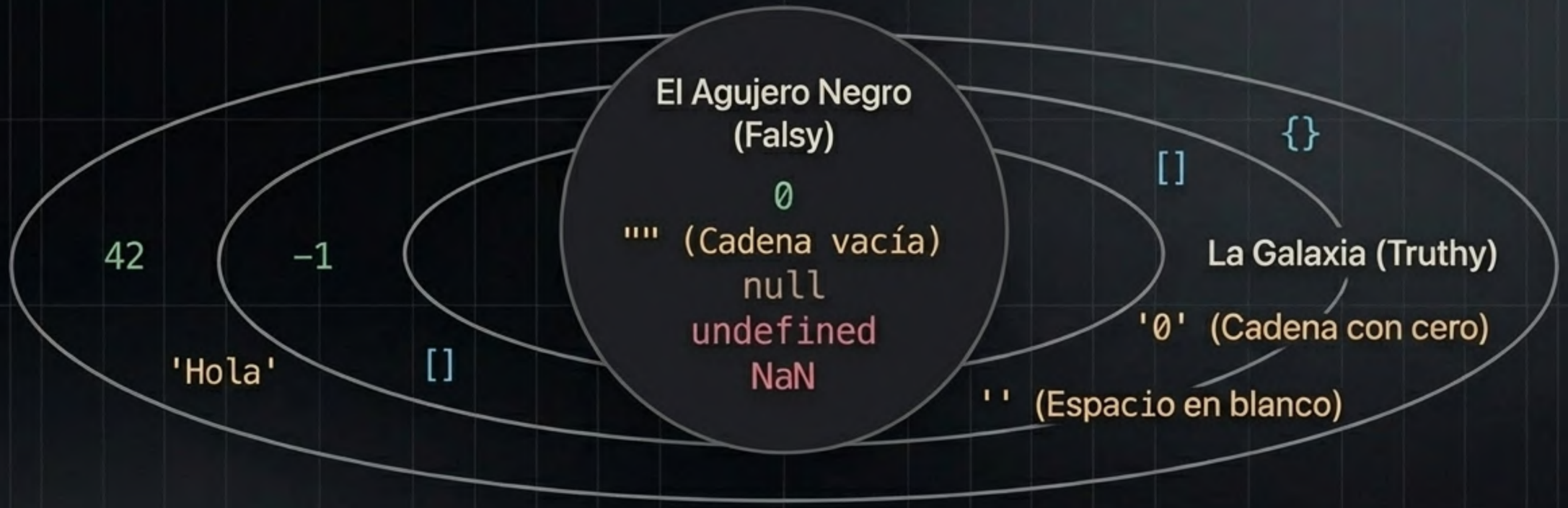
Prevención

Trata cualquier comparación matemática con estas variables con extrema precaución. Verifícalos de forma explícita o usa '==='.

NotebookLM

La Gravedad de la Verdad (Truthy y Falsy)

Conversión Booleana: Cuando JavaScript evalúa una expresión, la fuerza a convertirse en `true` o `false`.



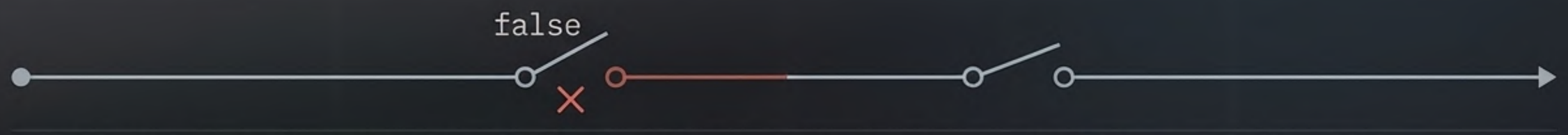
Cualquier otro valor en JavaScript, fuera del centro, es **true.**

NotebookLM

Los Guardianes Lógicos

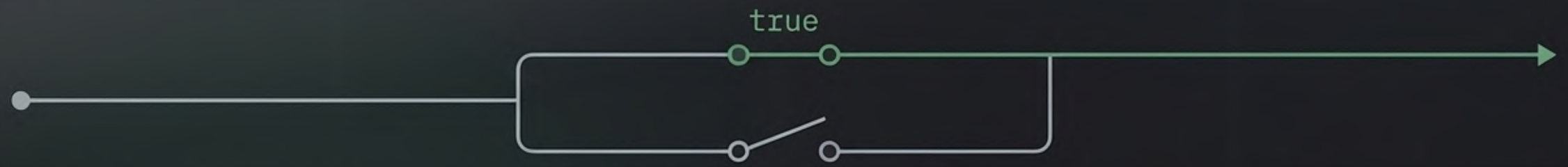
Evaluación de Cortocircuito (Short-circuit): Operadores que devuelven valores sin evaluar toda la línea.

El Operador AND (&&)



Evalúa de izquierda a derecha. Se detiene y devuelve el primer valor Falsy que encuentra.

El Operador OR (||)

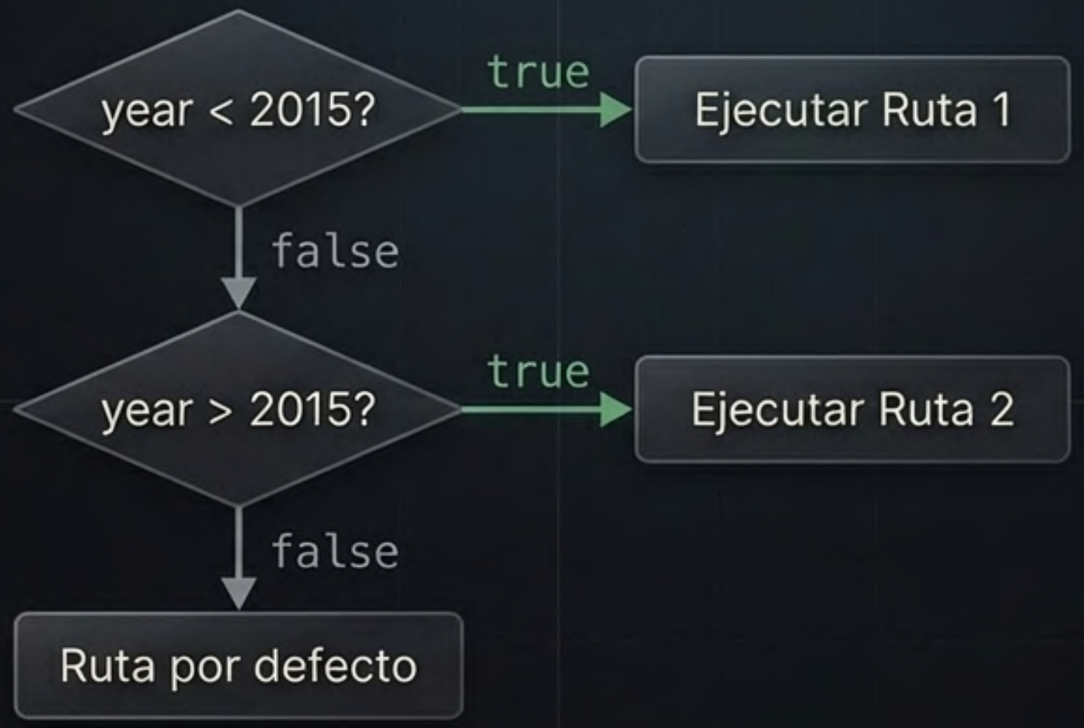


Evalúa de izquierda a derecha. Se detiene y devuelve el primer valor Truthy que encuentra.

NotebookLM

Rutas y Bifurcaciones

```
if (year < 2015) {
  // Ruta 1
} else if (year > 2015) {
  // Ruta 2
} else {
  // Ruta por defecto
}
```



Buena Práctica: Siempre envuelve tu bloque de código entre llaves {}, incluso si es una sola línea. Mejora la legibilidad.

NotebookLM

La Elección Elegante: El Operador Ternario '?'

El único operador de 3 partes. Asigna una variable dependiendo de una condición de forma condensada.

Bloque Tradicional

```
let accessAllowed;
if (age > 18) {
    accessAllowed = true;
} else {
    accessAllowed = false;
}
```

Sintaxis Ternaria

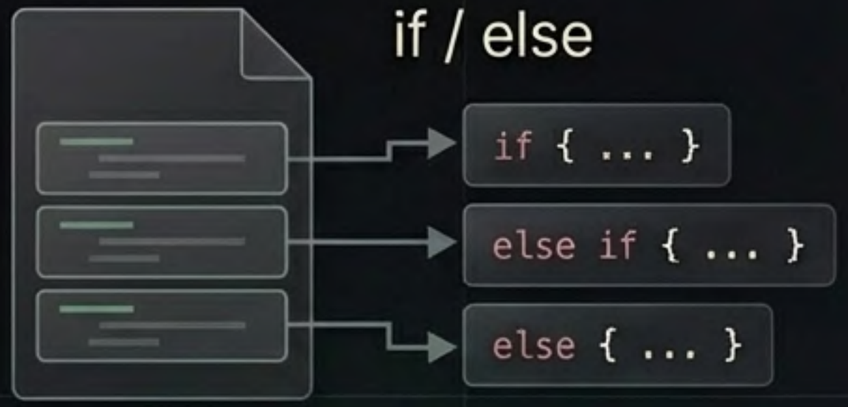
```
let accessAllowed = (age > 18) ? true : false;
```

condición operador valor_si_true separador valor_si_false

NotebookLM

Matrices de Decisión: ¿Cuál Herramienta Usar?

if / else



Usa para ejecutar acciones (ramas de código distintas). Su lectura vertical facilita el escaneo ocular.

Operador Ternario (?)

```
let result = (condition) ? valueIfTrue : valueIfFalse;
```

Usa exclusivamente para retornar y asignar un solo valor de manera elegante.

El Anti-Patrón (No recomendado)

```
let result = a ? b : c ? d : e;
```

Evita las secuencias múltiples de ternarios anidados. Aunque son técnicamente válidas, destruyen la legibilidad del código para otros ingenieros.

NotebookLM

Síntesis: El Circuito Completo

Combinando conversión, comparación matemática, lógica estricta y bifurcación en una sola línea maestra.

2. Juicio Estricto: Comprueba la igualdad sin coerción de tipos.

1. Conversión: El unario '+' convierte el string 'age' en número.

```
let status = (+age === 18 && balance % 2 === 0)
    ? 'Acceso' : 'Denegado';
```

3. Matemática Avanzada: Verifica si el saldo es perfectamente par.

4. El Guardián: Une ambas verdades; si la primera falla, hace cortocircuito.

5. La Bifurcación: Asigna elegantemente el resultado a la variable.

NotebookLM